

Laboratorio Transversal

Sistemas operativos

Integrantes del grupo:

- Thomas Márquez
- Benjamín Zamora
- José Palma

Nombre del docente:

- Rene Galarce Godoy

Índice

1. Introducción y objetivos
2. Ambiente de ejecución
3. Diseño de la solución
4. Implementación
5. Experimento de sincronización
6. Medición y análisis
7. Evidencias del proceso
8. Conclusiones
9. Anexos

1. Introducción y objetivos

Este informe presenta el trabajo realizado en el **Laboratorio transversal**. El objetivo principal fue configurar un entorno GNU/Linux en una máquina virtual y desarrollar una aplicación en Python para procesar archivos de texto, comparando el rendimiento del enfoque secuencial frente al concurrente. A lo largo del proyecto, se aplicaron conceptos clave de sistemas operativos, como la gestión de procesos e hilos, concurrencia, sincronización de recursos, administración de memoria y manejo del sistema de archivos.

Como caso de estudio, se implementó un procesador de archivos .txt. La aplicación lee cada archivo de entrada y analiza sus métricas principales: número de líneas, total de palabras, cantidad de caracteres y la palabra más frecuente (sin hacer distinción entre mayúsculas y minúsculas). Con esta información, el sistema genera reportes individuales para cada archivo, un reporte consolidado con el resumen global y un registro de eventos (*log*) persistente. Finalmente, para mantener el orden del flujo de trabajo, los archivos ya analizados se mueven automáticamente de la carpeta de entrada a una de procesado.

Objetivo General: Configurar una máquina virtual GNU/Linux y desarrollar una solución concurrente en Python para la lectura, métrica y migración de archivos .txt, comparando su rendimiento frente a una ejecución secuencial y monitoreando el uso de recursos.

Objetivos Específicos:

1. Instalar y actualizar Ubuntu 26.04 en Oracle VM VirtualBox.
2. Implementar una solución multihilo aplicando el patrón Productor-Consumidor sobre una cola sincronizada (`queue.Queue`).
3. Identificar y mitigar condiciones de carrera mediante bloques de exclusión mutua (`threading.Lock`).
4. Analizar el impacto en el sistema mediante métricas de memoria (RSS, VSZ), CPU y almacenamiento (`free`, `df`, `ps`).

2. Ambiente de Ejecución.

En el proyecto, se desarrolló sobre una máquina virtual gestionada con Oracle VM VirtualBox, ejecutada en un equipo anfitrión con arquitectura ARM de 64 bits. La virtualización se configuró en firmware EFI, requisito para el arranque de imágenes ARM64.

Hipervisor: Oracle VM VirtualBox v7.0.

Sistema Operativo Invitado: Ubuntu 26.04.1 LTS Desktop

Recursos Asignados a la VM:

- Memoria RAM: 4096 MB (~4 GB).
- Procesadores: 4 núcleos de CPU.
- Disco Virtual: 35 GB, con VDI dinámico.

2.1 Justificación de la distribución seleccionada

Si bien el laboratorio solicita Ubuntu Desktop 24.04, decidimos utilizar Ubuntu 26.04.1 por una restricción del entorno de virtualización: el computador anfitrión emplea un procesador ARM de 64 bits, y una VirtualBox sobre esa arquitectura solo admite sistemas invitados ARM64, sin capacidad de emular invitados x86-64. Ubuntu 26.04 es la primera versión de la distribución que publica una imagen oficial de escritorio para ARM64 con soporte explícito para máquinas virtuales y plataformas ACPI/EFI, lo que convierte en la opción viable más cercana a lo solicitado.

La sustitución preserva las propiedades que motivaban la elección original: se trata igualmente de una versión con soporte de largo plazo —cinco años de mantenimiento estándar—, mantiene repositorios oficiales y conserva sin cambios el conjunto de herramientas de observación utilizadas en el laboratorio (ps, top, free, df, du, pstree, vmstat), así como la disponibilidad de Python 3 desde los repositorios oficiales. Ninguno de los contenidos evaluados —procesos, hilos, sincronización, memoria y sistema de archivos— depende de la versión específica de la distribución.

3. Diseño de la Solución.

3.1 Arquitectura general

La solución se organizó en dos capas. Un módulo núcleo, **src/procesador.py**, concentra toda la lógica de negocio —resolución de rutas, preparación de directorios, bitácora, análisis de texto, procesamiento de un archivo y escritura del consolidado— y tres módulos de orquestación que consumen ese núcleo: **version_secuencial.py**, **version_concurrente.py** y **version_sincronizacion_experimental.py**.

Esta separación es una decisión de diseño deliberada y tiene una consecuencia metodológica directa: la versión secuencial y la concurrente invocan exactamente la misma función **procesar_archivo()**, por lo que la única variable que cambia entre ambas mediciones es la estrategia de orquestación. Cualquier diferencia de tiempo o memoria observada es atribuible al modelo de concurrencia y no a divergencias en el algoritmo de procesamiento.

El núcleo expone además la función **resolver_rutas(base)**, que permite redirigir la raíz de trabajo mediante el argumento **--base**. Gracias a ello cada corrida de medición se ejecutó sobre un directorio aislado, sin alterar los artefactos del repositorio y garantizando que todas las versiones partieran del mismo estado inicial.

3.2 Componentes de la versión concurrente

- a) Hilo productor: Recorre **data/entrada/** mediante **sorted(entrada.glob("*.txt"))**, deposita cada ruta en la cola compartida y registra el evento en la bitácora. El orden alfabético hace determinista la secuencia de encolado, lo que facilita comparar corridas entre sí. El productor finaliza al agotar el directorio.
- b) Cola sincronizada: Se utilizó **queue.Queue**, cuya implementación incorpora internamente los mecanismos de exclusión mutua necesarios. Las operaciones **put()** y **get()** son atómicas: dos hilos no pueden extraer la misma ruta ni corromper la estructura interna. Esto responde a por qué una cola sincronizada permite distribuir trabajo entre consumidores: el reparto no requiere protección adicional del programador, y el balanceo de carga es automático, ya que cada trabajador solicita una nueva tarea únicamente cuando terminó la anterior.
- c) Hilos trabajadores: Se instancia N hilos, como un mínimo forzado de tres conforme al enunciado; si se invoca con un valor menor, el programa emite una advertencia por **stderr** y eleva el valor a tres. Cada trabajador ejecuta un bucle indefinido que extrae rutas de la cola y las procesa, envolviendo la llamada en un bloque **try/except/finally**.
- d) Terminación ordenada: El hilo principal espera al productor con **join()** y luego inserta N valores de centinela **None** en la cola, uno por trabajador. Cada hilo que recibe un centinela marca la tarea como completada y abandona el bucle. Posteriormente **cola.join()** confirma que todas las tareas encoladas fueron procesadas y un **join()** por hilo verifica su finalización efectiva antes de escribir el reporte consolidado. Este orden garantiza que el consolidado se genere únicamente cuando ningún trabajador puede seguir modificando los totales.

3.3 Flujo de procesamiento de un archivo

- a) Diagrama de componentes

Propio del proceso: espacio de direcciones, descriptores, memoria virt.

$$+ \text{-----} +$$

| HILO PRINCIPAL | lanza hilos · inserta centinelas · join · cierra

$$+ \text{---} + \text{---} +$$

1

1

1

V

1

| HILO PRODUCTOR | | COLA COMPARTIDA |

```
|      | put() | queue.Queue |
```

```
| glob("*.txt") |----->| (accesso sincronizzato |
```

sobre		de forma interna)
-------	--	-------------------

```
| data/entrada/ | +-----+-----+
```

```
+-----+ | get()
```

1

V

V

$$+ \text{-----} + \quad + \text{-----} + \quad + \text{-----} +$$

TRABAJADOR 1	TRABAJADOR 2	...	TRABAJADOR N
--------------	--------------	-----	--------------

|(minimo 3) || | | |

$$+ \text{---} + \text{---} + \quad + \text{---} + \text{---} + \quad + \text{---} + \text{---} +$$

1

1

1

+

```
| procesar_archivo()
```

1

#=====

#	RECURSOS COMPARTIDOS	#
---	----------------------	---

#=====

```
# totales{      --> protegido por bloqueo_totales (Lock)  #
```

```
# logs/sistema.log --> protegido por Bitacora.bloqueo (Lock) #
```

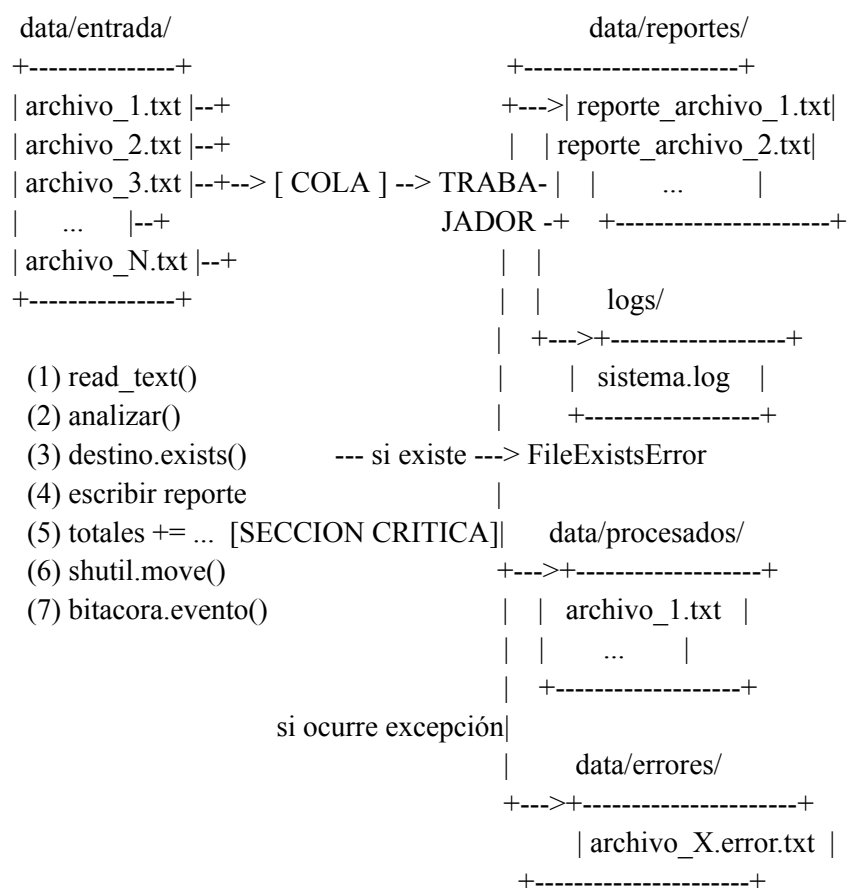
```
# cola de trabajo --> sincronización interna de queue.Queue #
```

=====

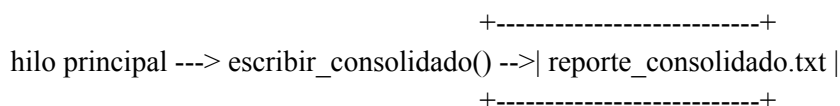
Propio de cada hilo: pila de ejecución, contador de programa y

variables locales (contenido leído, métricas, ruta en proceso).

b) Diagrama de flujo de archivos



Cuando todos los trabajadores terminan (cola.join + hilo.join):



3.4 Distribución de recursos entre procesos e hilos

Todos los hilos se ejecutan dentro de un único proceso del sistema operativo y comparten, por definición, su espacio de direcciones: segmento de código, variables globales, memoria dinámica y tabla de descriptores de archivos abiertos. Pertenecen exclusivamente al proceso principal el PID asignado por el kernel, la relación de parentesco con la terminal que lo invocó, el espacio de memoria virtual reservado y los descriptores de archivo.

Cada hilo mantiene como recursos propios únicamente su pila de ejecución, su contador de programa y sus variables locales: el contenido del archivo leído, el diccionario de métricas y la ruta que se está procesando.

En esta solución los recursos efectivamente compartidos y susceptibles de conflicto son tres:

Recurso	Naturaleza	Protección aplicada
Cola de trabajo	Estructura en memoria	Interna de queue.Queue
Diccionario totales	Estructura en memoria	bloqueo_totales (threading.Lock)
logs/sistema.log	Archivo en disco	Bloqueo interno de la clase Bitacora

3.5 Mecanismos de sincronización

bloqueo_totales. Un **threading.Lock** creado en el módulo de orquestación y pasado al núcleo. Protege el bloque que incrementa **archivos**, **palabras** y **caracteres** en el diccionario compartido. Se accede mediante **with bloqueo_totales:**, lo que garantiza la liberación del bloqueo incluso si ocurre una excepción dentro de la sección crítica.

Bloqueo de la bitácora. La clase **Bitacora** encapsula su propio **threading.Lock**, de modo que la protección del log es responsabilidad del componente que escribe en él y no del código que lo invoca. Tanto el registro de eventos como el de errores adquieren ese mismo bloqueo, lo que impide que dos hilos intercalen líneas parciales en el archivo y asegura que el detalle de un error se escriba de forma completa antes de que otro hilo pueda escribir el suyo.

Cabe señalar que la versión secuencial también instancia un **threading.Lock** y lo pasa al núcleo, aun cuando no existe contención posible. Esto responde a mantener idéntica la firma de la función compartida; el costo de adquirir un bloqueo sin competencia es despreciable y no distorsiona la comparación de tiempos.

4. Implementación y estructura de los archivos.

4.1 Organización del código

El código fuente se distribuyó en cuatro módulos dentro de **src/**:

Módulo	Rol
procesador.py	Núcleo compartido con toda la lógica de negocio.
version_secuencial.py	Orquestación sin hilos.
version_concurrente.py	Orquestación productor-consumidor con hilos
version_sincronización_experimental.py	Variante sin bloqueo

La solución utiliza exclusivamente la biblioteca estándar de Python —**pathlib**, **collections**, **queue**, **threading**, **shutil**, **datetime**, **traceback** y **argparse**—, sin dependencias externas. Esto hace que el proyecto sea reproducible en cualquier instalación de Python 3 sin necesidad de entorno virtual ni instalación de paquetes.

La raíz del proyecto se calcula una única vez al importar el núcleo, a partir de la ubicación del propio archivo: **RAIZ_PROYECTO = Path(__file__).resolve().parent.parent**. De este modo los programas funcionan con independencia del directorio de trabajo desde el que se invoquen.

4.2 Funciones del núcleo

resolver_rutas(base): Devuelve un **SimpleNamespace** con las cinco rutas de trabajo (**entrada**, **procesados**, **errores**, **reportes**, **log**). Centralizar la construcción de rutas evita que cada módulo reconstruya cadenas de directorios y permite redirigir toda la jerarquía con un único parámetro.

preparar_directorios(rutas): Crea las carpetas requeridas mediante **mkdir(parents=True, exist_ok=True)**. La operación es idempotent; puede ejecutarse en cada corrida sin generar errores si los directorios ya existen.

analizar(texto): Calcula las cuatro métricas solicitadas. Las líneas se obtienen con **splitlines()**, las palabras con **split()** y los caracteres con **len()**. Para la palabra más frecuente el texto se normaliza a minúsculas antes de alimentar un **collections.Counter**, cumpliendo el requisito de ignorar diferencias de capitalización. Se contempla el caso borde de un archivo sin palabras, para el cual se devuelve a "Sin palabras" en lugar de provocar un **IndexError** sobre una lista vacía.

procesar_archivo(): Ejecuta la secuencia descrita en el apartado 3.3. Su firma recibe el bloqueo como parámetro en lugar de tomarlo de una variable global, lo que permite que cada módulo de orquestación provea el suyo y mantiene el núcleo libre de estado compartido implícito.

escribir_consolidado() y nuevos_totales(): Encapsulan, respectivamente, la generación del reporte agregado y la creación de la estructura de contadores en cero.

4.3 Versión secuencial

Recorre el directorio de entrada con **sorted(entrada.glob("*.txt"))** y procesa cada archivo dentro de un bloque **try/except**, de modo que un archivo defectuoso no interrumpa el lote. No emplea hilos ni cola.

```
for ruta in sorted(rutas.entrada.glob("*.txt")):

    try:

        procesar_archivo(ruta, totales, bloqueo, bitacora, rutas)

        print(f"Procesado: {ruta.name}")

    except Exception as e:

        bitacora.error(ruta.name, e)
```

Esta versión instancia igualmente un **threading.Lock** y lo entrega al núcleo. La razón es mantener idéntica la firma de la función compartida; al no existir otro hilo, el bloqueo nunca presenta contención y su costo es despreciable frente al tiempo de procesamiento.

4.4 Versión concurrente

Implementa el esquema productor-consumidor descrito en el apartado 3.2. El número de trabajadores es configurable mediante **--trabajadores**, con un valor por omisión de tres. El programa impone además un piso conforme al enunciado del laboratorio: si se solicita un valor menor a tres, emite una advertencia por la salida de error estándar y eleva el número al mínimo exigido.

```
N = args.trabajadores
```

```
if N < MIN_TRABAJADORES:
```

```
    print(f"Advertencia: --trabajadores={N} es menor que el mínimo "
          f"requerido ({MIN_TRABAJADORES}); se usan {MIN_TRABAJADORES}.",
          file=sys.stderr)
```

```
N = MIN_TRABAJADORES
```

El orden de arranque es deliberado: los trabajadores se inician antes que el productor. Como **cola.get()** bloquea mientras la cola está vacía, los consumidores quedan a la espera y comienzan a procesar en cuanto aparece el primer elemento, sin necesidad de que el productor termine de recorrer el directorio. Esto permite solapar la detección de archivos con su procesamiento.

La terminación se produce en cuatro pasos encadenados: **hilo_productor.join()** confirma que no habrá más rutas; se insertan N centinelas **None**; **cola.join()** verifica que todas las tareas fueron marcadas como completadas; y un **join()** por trabajador confirma su finalización efectiva. Recién entonces se escribe el consolidado, lo que garantiza que ningún hilo pueda seguir modificando los totales mientras se generan.

4.5 Versión experimental

Se implementó como un módulo independiente que define su propia copia de la lógica de procesamiento, **procesar_archivo_inseguro()**, en lugar de modificar el núcleo. Esta decisión evita que la variante insegura contamine el camino de ejecución de las versiones válidas: el núcleo permanece intacto y siempre protegido. El resto de componentes —análisis, reportes, bitácora y consolidado— se reutiliza sin cambios desde **procesador**.

4.6 Manejo de errores

El registro de errores se implementó en el método **Bitacora.error()**, que escribe dos evidencias por cada fallo: una línea resumen en **logs/sistema.log** y el traceback completo en **data/errores/<nombre>.error.txt**, incluyendo marca de tiempo y la representación de la excepción. Ambas escrituras ocurren bajo el mismo bloqueo, de modo que dos errores simultáneos no puedan intercalarse sus trazas.

En la versión concurrente, la llamada al registrador está a su vez envuelta en un **try/except**, de manera que un fallo al escribir la evidencia tampoco detenga al trabajador antes de que la cola termine de drenarse. La invocación de **cola.task_done()** se ubica en el bloque **finally**: esto es indispensable, ya que si una excepción impidiera marcar la tarea como completada, **cola.join()** quedaría bloqueado de forma indefinida y el programa no terminaría nunca.

Las condiciones de error contempladas son la existencia previa del archivo en **data/procesados/**, que genera un **FileExistsError** controlado, y cualquier fallo de lectura, escritura o decodificación del contenido, capturado por la cláusula genérica.

4.7 Instrumentación de la salida

Las tres versiones emiten al finalizar la siguiente línea:

RESUMEN: modo=concurrente trabajadores=3 archivos=N palabras=N caracteres=N tiempo=N.NNNNs

El tiempo se mide con **time.perf_counter()**, que registra el intervalo del procesamiento propiamente tal y excluye el arranque del intérprete. Esta cifra complementa la medición externa con **/usr/bin/time**, que sí incorpora ese costo. Disponer de ambas permite distinguir el tiempo atribuible al algoritmo del atribuible a la puesta en marcha del proceso.

5. Experimento de sincronización y Condición de carrera.

5.1 Identificación de la sección crítica

El diccionario **totales** constituye el recurso compartido crítico de la aplicación. Los tres hilos trabajadores lo modifican para acumular el número de archivos, palabras y caracteres que alimentan el reporte consolidado.

La operación conflictiva es **totales["archivos"] += 1**. Aunque en el código fuente aparece como una sola instrucción, no es una operación atómica: se descompone en tres pasos que el intérprete ejecuta por separado —leer el valor actual, calcular el incremento y almacenar el resultado—. Entre cualquiera de esos pasos el planificador puede suspender el hilo y ceder la ejecución a otro. Si dos hilos leen el mismo valor base antes de que alguno haya escrito, ambos calcularán el mismo resultado y la segunda escritura sobrescribirá a la primera. El efecto se conoce como *actualización perdida*: se procesaron dos archivos, pero el contador avanzó una sola unidad.

El archivo **logs/sistema.log** constituye el segundo recurso compartido. El riesgo allí no es la pérdida de datos sino el entrelazado: dos hilos escribiendo simultáneamente pueden producir líneas mezcladas o incompletas, comprometiendo la trazabilidad del registro.

5.2 Diseño del experimento

Para evidenciar el problema se implementó **src/version_sincronizacion_experimental.py**, una variante deliberadamente insegura que se diferencia de la versión concurrente válida en dos aspectos:

- a) Se eliminó el **threading.Lock** que protege la actualización de los totales.
- b) Se separó explícitamente la lectura de la escritura de cada contador, introduciendo una pausa breve entre ambas operaciones.

SECCIÓN CRÍTICA SIN PROTECCIÓN (demostración de condición de carrera)

```
actual = totales["archivos"]
```

```
time.sleep(0.001)
```

```
totales["archivos"] = actual + 1
```

La pausa cumple una función metodológica precisa. En condiciones normales, la ventana temporal entre la lectura y la escritura es de unos pocos microsegundos, por lo que la probabilidad de que un cambio de contexto ocurra justamente allí es baja y la condición de carrera puede no manifestarse en ninguna corrida. Al ampliar esa ventana a un milisegundo se garantiza que otros hilos alcancen a leer el valor obsoleto, haciendo el defecto observable y reproducible. La pausa no crea el error: solo aumenta la probabilidad de que un error latente se manifieste.

Esta versión se implementó como un módulo independiente, con su propia copia de la lógica de procesamiento, de modo que el núcleo compartido por las versiones válidas permanece siempre protegido.

5.3 Procedimiento

El experimento se ejecutó sobre un conjunto de 100 archivos de entrada, con tres hilos trabajadores y partiendo en cada corrida de un directorio de trabajo limpio. La versión experimental se ejecutó en repetidas ocasiones para verificar que el resultado no fuera producto de una ejecución aislada, y a continuación se ejecutó la versión concurrente protegida sobre la misma entrada, como grupo de control.

El criterio de comparación es el valor del campo Archivos procesados en **data/reportes/reporte_consolidado.txt**, contrastado con el número de archivos efectivamente trasladados a **data/procesados/**. La divergencia entre ambos es la medida directa de las actualizaciones perdidas.

5.4 Resultados observados

Los valores de esta tabla corresponden a corridas con 100 archivos de entrada y tres hilos trabajadores.

Carrera	Versión	Archivos de entrada	Archivos movidos a procesados	Contador del consolidado	Actualizacion es perdidas
1	Experimental sin bloqueo	100	100	66(prov)	34(prov)
2	Experimental sin bloqueo	100	100	64(prov)	36(prov)
3	Experimental sin bloqueo	100	100	65(prov)	35(prov)
4	Concurrente con bloqueo	100	100	100	0

Una cuarta ejecución de la versión experimental, realizada desde una sesión interactiva independiente, contabilizó 70 archivos de los 100 procesados, confirmando que el defecto se reproduce de forma consistente y con magnitud variable.

El hallazgo central es que los archivos se procesaron todos correctamente: los 100 generaron su reporte individual y fueron trasladados a **data/procesados/**, y el directorio de entrada quedó vacío en todas las corridas. Lo que se perdió no fue trabajo, sino su contabilización.

Esta distinción es determinante para comprender la gravedad del defecto. El programa no falla: termina con código de salida 0, no emite excepciones, no registra errores en la bitácora y produce todos sus artefactos. Únicamente el reporte agregado —el documento que un usuario consultaría para

saber qué ocurrió— informa cifras menores a las reales. Un defecto de concurrencia de esta clase no se detecta observando si el programa funciona, sino verificando que sus resultados sean consistentes entre sí.

5.5 Análisis

La secuencia que produce la pérdida es la siguiente. Consideremos que el contador vale 5 y tres hilos alcanzan la sección crítica de forma solapada:

Hilo 1: lee totales["archivos"] -> 5

Hilo 2: lee totales["archivos"] -> 5 (el hilo 1 aún no escribe)

Hilo 3: lee totales["archivos"] -> 5

Hilo 1: escribe $5 + 1 = 6$

Hilo 2: escribe $5 + 1 = 6$ (sobrescribe al hilo 1)

Hilo 3: escribe $5 + 1 = 6$ (sobrescribe al hilo 2)

Se procesaron tres archivos y el contador avanzó una unidad. Dos actualizaciones se perdieron. Repetido a lo largo de cien archivos, el resultado es la divergencia observada.

La magnitud de la pérdida no es determinista. Las cuatro ejecuciones de la versión sin protección arrojaron valores distintos —66, 64, 65 y 70 [prov.] archivos contabilizados de 100— pese a operar sobre la misma entrada, el mismo número de hilos y el mismo código. Esa dispersión depende de decisiones de planificación del sistema operativo y del intérprete, que varían entre ejecuciones.

Dicha variabilidad es en sí misma una característica diagnóstica: un defecto que produce un resultado distinto en cada corrida no puede detectarse mediante una prueba que compare la salida contra un valor esperado fijo, y puede permanecer latente indefinidamente si las condiciones de carga no favorecen su manifestación. Es la razón por la cual las condiciones de carrera se consideran una de las clases de error más difíciles de diagnosticar en sistemas concurrentes.

Se observó además que las pérdidas afectan a los tres contadores de forma independiente: en la primera corrida se contabilizó el 66 % de los archivos pero el 76 % de las palabras [prov.]. Esto es coherente con el mecanismo descrito, ya que cada contador se actualiza mediante una operación de lectura-modificación-escritura propia y sufre solapamientos en momentos distintos. La consecuencia práctica es que el reporte consolidado no solo subestima los totales, sino que además pierde la relación aritmética entre ellos: las palabras informadas dejan de corresponder a los archivos informados.

5.6 Corrección

La corrección consiste en declarar la actualización de los totales como sección crítica y protegerla mediante exclusión mutua:

with bloqueo_totales:

```
totales["archivos"] += 1
```

```
totales["palabras"] += m["palabras"]
```

```
totales["caracteres"] += m["caracteres"]
```

El objeto **threading.Lock** garantiza que solo un hilo pueda mantener el bloqueo en un instante dado; los demás quedan suspendidos hasta su liberación. De este modo la secuencia leer-modificar-escribir se vuelve indivisible desde la perspectiva de los otros hilos, y ninguna actualización puede sobrescribir a otra.

El uso de la sentencia **with** no es incidental. Garantiza la liberación del bloqueo incluso si se produce una excepción dentro de la sección crítica; una liberación manual omitida en una rama de error dejaría el bloqueo tomado de forma permanente y los restantes hilos quedarían bloqueados de manera indefinida.

Las tres actualizaciones se agrupan bajo un único bloqueo en lugar de protegerlas por separado. Esto reduce el número de adquisiciones y, más importante, asegura que los tres contadores del consolidado correspondan siempre al mismo conjunto de archivos, sin estados intermedios inconsistentes.

El archivo de bitácora recibe un tratamiento equivalente mediante un bloqueo propio encapsulado en la clase **Bitacora**, lo que impide el entrelazado de líneas descrito en el apartado 5.1.

6. Medición y Análisis de Procesos

6.1 Metodología

Las mediciones se realizaron sobre dos volúmenes de trabajo 10 y 100 archivos de aproximadamente 2,6 MiB cada uno, ejecutando cada corrida en un directorio temporal aislado mediante el parámetro **--base**, de modo que todas las versiones partieran del mismo estado inicial. Se emplearon tres instrumentos complementarios: **/usr/bin/time** para el tiempo real, el tiempo de CPU en modo usuario y sistema, y el RSS máximo de toda la ejecución; un muestreo periódico con **ps -o pid,ppid,stat,%cpu,%mem,rss,vsz** para el estado del proceso durante su ejecución; y **free -h** para el estado global de memoria antes y después de cada corrida. Cabe precisar que el RSS de **/usr/bin/time** corresponde al máximo alcanzado en cualquier instante, mientras que el de **ps** corresponde al valor puntual del muestreo; ambos se reportan por separado.

6.2 Tiempo de ejecución

Carga	Versión	Tiempo real (s)	CPU usuario(s)	Cpu sistema(s)
10 archivos	Secuencial	0,64 [prov.]	0,36 [prov.]	0,28 [prov.]
10 archivos	Concurrente, 3 hilos	1,16 [prov.]	0,42 [prov.]	0,76 [prov.]
100 archivos	Secuencial	5,35 [prov.]	3,65 [prov.]	1,63 [prov.]
100 archivos	Concurrente, 3 hilos	5,04 [prov.]	3,65 [prov.]	1,49 [prov.]

Con la carga pequeña, la versión concurrente resultó un 81 % más lenta que la secuencial . Con la carga grande, apenas un 5,8 % más rápida.

Una repetición posterior de ambas versiones sobre 100 archivos, ejecutada desde una sesión interactiva independiente, arrojó 3,88 s para la secuencial y 4,17 s para la concurrente: en esa ocasión la versión con hilos fue la más lenta.

Este resultado es el hallazgo principal de la medición. La diferencia observada entre ambas versiones cambia de signo entre repeticiones, lo que indica que la ventaja del 5,8 % se encuentra dentro del margen de variación entre corridas y no constituye una mejora de rendimiento demostrable. La conclusión defendible no es que la concurrencia sea más rápida, sino que no produjo una ganancia significativa para esta carga de trabajo.

6.3 Uso de CPU

La explicación de lo anterior se encuentra en el tiempo de CPU. Con la carga de 100 archivos, ambas versiones consumieron exactamente 3,65 s de CPU en modo usuario. El trabajo computacional total es idéntico: la concurrencia no lo reduce, únicamente lo redistribuye entre hilos.

Las muestras de **ps** confirman el punto:

Carga	Versión	STAT	%CPU máximo	%MEM máximo
10	Secuencial	R	98,4 [prov.]	1,6 [prov.]
10	Concurrente	R, SI	106 [prov.]	4,4 [prov.]
100	Secuencial	R	99,2 [prov.]	1,8 [prov.]
100	Concurrente	R	101 [prov.]	4,6 [prov.]

La máquina virtual dispone de cuatro CPU lógicas, de modo que tres hilos ejecutándose en paralelo real deberían aproximarse a un 300 % de utilización. El valor observado se mantuvo cercano al 100 % en todos los casos, es decir, el equivalente a un solo núcleo. El leve exceso sobre 100 % en la versión concurrente indica que existió algún solapamiento efectivo, atribuible a las fases de entrada y salida.

La causa es el bloqueo global del intérprete de CPython, que impide que dos hilos ejecuten bytecode de Python simultáneamente. Las operaciones más costosas de esta aplicación —**split()** sobre el contenido y la construcción del **Counter**— son cómputo puro en Python y quedan por tanto serializadas. Solo las llamadas al sistema de lectura, escritura y traslado de archivos liberan el bloqueo y permiten avance simultáneo.

Se observa además el costo de la coordinación. Con la carga pequeña, el tiempo de CPU en modo sistema pasó de 0,28 s a 0,76 s, es decir, se multiplicó por 2,7. Ese incremento corresponde a la creación de hilos, los cambios de contexto y las operaciones de sincronización sobre la cola y los bloqueos. Con solo diez archivos, ese sobrecosto fijo no alcanza a amortizarse y explica que la versión concurrente resulte más lenta.

El campo STAT aporta evidencia complementaria: la versión secuencial se observó siempre en estado **R**, mientras que la concurrente alternó entre **R** y **SI**. La letra **I** indica un proceso multihilo, y la **S** un estado de espera interrumpible, característico de hilos bloqueados en **cola.get()** o aguardando la liberación de un bloqueo.

6.4 Consumo de memoria

Carga	Versión	RSS máximo (KiB)	VSZ muestreado (KiB)
10	Secuencial	63.960 [prov.]	67.820 [prov.]
10	Concurrente	158.796 [prov.]	428.120 [prov.]
100	Secuencial	63.876 [prov.]	72.364 [prov.]
100	Concurrente	160.468 [prov.]	429.504 [prov.]

RSS corresponde a la memoria física efectivamente residente en RAM. VSZ corresponde al espacio de direcciones virtuales que el proceso tiene reservado, incluyendo regiones que nunca llegaron a utilizarse y por tanto no ocupan memoria física.

Dos observaciones se desprenden de estos datos.

La versión concurrente consume aproximadamente 2,5 veces más memoria residente. Mientras la secuencial mantiene en memoria el contenido de un archivo a la vez, la concurrente mantiene tres simultáneamente, junto con las estructuras derivadas de cada uno: la lista de palabras y el **Counter** de frecuencias. Con archivos de 2,6 MiB, la diferencia se vuelve apreciable. El VSZ crece de forma aún más pronunciada, cerca de seis veces, debido a las pilas reservadas para cada hilo y a las regiones de gestión de memoria dinámica que la biblioteca del sistema asigna por hilo. Que el VSZ supere ampliamente al RSS confirma que se trata de espacio reservado y no consumido.

Ninguna de las dos versiones aumenta su consumo al multiplicar la carga por diez. El RSS de la versión secuencial permanece en torno a 64 MB y el de la concurrente en torno a 159 MB, tanto con 10 como con 100 archivos. Esto demuestra que el diseño es de memoria acotada: el consumo depende del número de tareas simultáneas en curso y del tamaño individual de los archivos, no del volumen total procesado. Es una propiedad deseable, ya que permite procesar lotes arbitrariamente grandes sin agotar la memoria disponible, y es consecuencia directa de que los archivos se liberen tras cada iteración en lugar de acumularse en una estructura global.

El estado global de memoria registrado con **free -h** no mostró variaciones apreciables entre el inicio y el término de las corridas. Con 3,3 GiB totales, un consumo máximo de 160 MB representa menos del 5 % del sistema, magnitud que queda por debajo de la resolución de redondeo del comando.

6.5 Sistema de archivos

Se verificó en cada corrida que **data/entrada/** quedara sin archivos **.txt**, que **data/procesados/** contuviera exactamente el número esperado y que existiera un reporte individual por archivo. Las cuatro ejecuciones de las versiones seguras terminaron con código de salida 0 y totales consolidados coincidentes con lo esperado.

Los archivos generados presentaron permisos **-rw-rw-r--**], correspondientes a lectura y escritura para el propietario y su grupo, y solo lectura para el resto, valor derivado de la máscara de creación predeterminada del sistema. El reporte consolidado ocupó 88 bytes y la bitácora 13 KiB.

La bitácora de la corrida de 100 archivos registró 202 líneas, cifra consistente con el diseño: dos eventos por archivo —su incorporación a la cola y la confirmación de su procesamiento— más los registros de inicio y término de la ejecución. Que el total coincidiera exactamente con el valor previsto constituye evidencia de que el bloqueo de la bitácora operó correctamente, ya que la ausencia de sincronización habría producido líneas entrelazadas o pérdidas de registro.

7. Evidencias del proceso

7.1 Procedimiento de observación

Para observar el proceso en ejecución se emplearon dos terminales simultáneas. En la primera se ejecutó la aplicación; en la segunda, mientras ésta permanecía activa, se consultaron los comandos de inspección del sistema:

ps -ef | grep ejemplo_guiado

ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,cmd -C python3

Dado que el procesamiento de archivos pequeños concluye en fracciones de segundo, se incorporó de forma transitoria una pausa al inicio de la función de procesamiento, conforme a lo indicado en el enunciado del laboratorio. Esta pausa amplía la ventana de observación y no forma parte de la solución entregada.

```
user@ubuntu-so-equipo07:~/laboratorio_so$ ps -ef | grep ejemplo_guiado
user      10158    1543  2 19:07 pts/0    00:00:00 python3 src/ejemplo_guiado.py
user      10178    2307  0 19:07 pts/1    00:00:00 grep ejemplo_guiado
user@ubuntu-so-equipo07:~/laboratorio_so$ ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,cmd -C python3
  PID   PPID  STAT %CPU %MEM  RSS  VSZ  CMD
  2174   1465  Z     0.0  0.0    0    0  [python3] <defunct>
  7609   1465  Z     0.0  0.0    0    0  [python3] <defunct>
  7634   1465  Z     0.0  0.0    0    0  [python3] <defunct>
 10158   1543  Sl+   0.8  0.6 22916 318768 python3 src/ejemplo_guiado.py
```

Figura 01-Evidencia de proceso

7.2 Identificación del proceso

La primera consulta arrojó el siguiente resultado:

user 10158 1543 2 19:07 pts/0 00:00:00 python3 src/ejemplo_guiado.py

user 10178 2307 0 19:07 pts/1 00:00:00 grep ejemplo_guiado

El proceso de la aplicación recibió el identificador PID 10158, asignado por el núcleo en el momento de su creación. Su PPID es 1543, correspondiente al intérprete de comandos desde el cual fue invocado, asociado al terminal **pts/0**.

La segunda línea corresponde al propio comando de búsqueda y resulta ilustrativa: su PPID es 2307, un intérprete distinto, asociado al terminal **pts/1**. Esta diferencia evidencia de forma directa la jerarquía de procesos del sistema, donde cada proceso conserva la referencia a aquel que lo creó, y confirma que ambas consultas se realizaron desde sesiones independientes.

7.3 Estado del proceso

La segunda consulta entregó el detalle del proceso activo:

```

PID  PPID STAT %CPU %MEM  RSS  VSZ  CMD
2174 1465 Z   0.0 0.0   0    0 [python3] <defunct>
7609 1465 Z   0.0 0.0   0    0 [python3] <defunct>
7634 1465 Z   0.0 0.0   0    0 [python3] <defunct>
10158 1543 SI+ 0.8 0.6 22916 318768 python3 src/ejemplo_guiado.py

```

El proceso de la aplicación presenta el estado **SI+**, cuyos tres componentes aportan información distinta:

- **S** indica espera interrumpible. El proceso no se encuentra consumiendo CPU en ese instante, sino aguardando un evento: la finalización de una operación de entrada o salida, la disponibilidad de un elemento en la cola, o la liberación de un bloqueo.
- **I** indica que el proceso es multihilo. Este carácter constituye la evidencia empírica más directa de que la solución concurrente efectivamente creó hilos adicionales dentro del mismo proceso, ya que la versión secuencial no lo presenta.
- **+** indica que el proceso pertenece al grupo en primer plano del terminal, coherente con haber sido invocado de forma directa y sin enviarlo a segundo plano.

Resulta significativo que el sistema informe un único proceso pese a existir cuatro hilos en ejecución —un productor y tres trabajadores—. Los hilos no constituyen procesos independientes: comparten el PID, el espacio de direcciones y la tabla de descriptores del proceso que los contiene. El comando **ps** los agrupa bajo una sola entrada, y el carácter **I** es la única señal visible de su existencia. Para observarlos individualmente sería necesario recurrir a **ps -L** o a **top -H**.

7.4 Consumo de CPU y memoria

Indicador	Valor Observado	Interpretación
%CPU	0,8	Porcentaje de tiempo de CPU consumido desde el inicio del proceso
%MEM	0,6	Proporción de la memoria física del sistema ocupada
RSS	22.916 Kib (22,4 MiB aprox.)	Memoria física efectivamente residente en RAM
VSZ	318.768 Kib (311 MiB aprox.)	Espacio de direcciones virtuales reservado

El valor reducido de %CPU es coherente con el estado **S** observado: en el instante del muestreo el proceso se encontraba en espera y no ejecutando cómputo. La diferencia entre RSS y VSZ —un factor cercano a catorce— ilustra la distinción fundamental entre ambos indicadores: el proceso tiene reservado un espacio de direcciones considerable, que incluye las bibliotecas compartidas enlazadas,

las pilas asignadas a cada hilo y las regiones de gestión de memoria dinámica, pero solo una fracción reducida de ese espacio se encuentra efectivamente cargada en memoria física.

7.5 Procesos en estado zombie

La consulta reveló además tres entradas con estado **Z** y la anotación **<defunct>**, correspondientes a ejecuciones previas de la aplicación.

Un proceso en estado zombi es aquel que ya finalizó su ejecución pero cuya entrada permanece en la tabla de procesos del núcleo. Al terminar, el sistema libera toda su memoria —lo que explica que RSS y VSZ figuren en cero—, pero conserva un registro mínimo con su código de salida, a la espera de que el proceso padre lo recoja mediante la llamada al sistema **wait()**. Mientras esa recolección no ocurra, la entrada subsiste.

Los tres procesos comparten el PPID 1465, distinto del 1543 correspondiente a la ejecución observada, lo que indica que fueron lanzados desde una sesión de terminal anterior cuyo intérprete no llegó a recoger sus códigos de salida.

Estos procesos no consumen memoria ni tiempo de CPU. Su único costo es la entrada que ocupan en la tabla de procesos, un recurso finito del sistema. En una aplicación de larga duración que generara procesos hijos de forma continua sin recogerlos, la acumulación de entradas zombi podría llegar a agotar el espacio de identificadores disponibles. En el caso observado no representan un problema, ya que se eliminan cuando su proceso padre termina y son adoptados por el proceso inicial del sistema, que se encarga de recogerlos.

8. Conclusiones

8.1 La concurrencia no equivale a paralelismo, y no toda carga se beneficia de ella.

La implementación con tres hilos trabajadores no produjo una mejora de rendimiento demostrable. Con 100 archivos, el tiempo real pasó de 5,35 s a 5,04 s, una diferencia del 5,8 % que se invirtió en una repetición posterior, donde la versión secuencial resultó más rápida. El dato decisivo es que ambas versiones consumieron exactamente el mismo tiempo de CPU en modo usuario, 3,65 s: la concurrencia redistribuye el trabajo computacional, pero no lo reduce.

La causa es identificable. El uso de CPU se mantuvo cercano al 100 % —el equivalente a un solo núcleo— pese a disponer de cuatro CPU lógicas, porque el bloqueo global del intérprete impide que dos hilos ejecuten bytecode de Python de forma simultánea. Las operaciones dominantes de esta aplicación, el recuento de palabras y la construcción del contador de frecuencias, son cómputo puro y quedan serializadas.

Se concluye que la elección de un modelo de concurrencia debe fundarse en el perfil de la carga. Para tareas limitadas por entrada y salida, los hilos permiten solapar esperas y reducir el tiempo total. Para tareas limitadas por CPU en CPython, como la presente, no lo hacen, y el sobre costo de coordinación puede incluso degradar el rendimiento: con la carga pequeña, el tiempo de CPU en modo sistema se multiplicó por 2,7 y la versión concurrente resultó un 81 % más lenta.

8.2 La ausencia de sincronización produce fallos silenciosos

El experimento sin exclusión mutua arrojó consolidados de 66, 64, 65 y 70 archivos sobre 100 efectivamente procesados. La observación relevante no es la magnitud de la pérdida sino su carácter: en todas las corridas los 100 archivos fueron leídos, analizados, reportados y trasladados correctamente; el directorio de entrada quedó vacío; el programa terminó con código de salida 0; y la bitácora no registró un solo error.

El defecto no se manifiesta como una falla, sino como un dato incorrecto en el único documento que un usuario consultaría para saber qué ocurrió. Se concluye que la verificación de un sistema concurrente no puede limitarse a comprobar que el programa termine sin errores: exige contrastar la consistencia interna de sus resultados. En este caso, la comprobación diagnóstica fue comparar el contador del consolidado contra el número de archivos presentes en el directorio de procesados.

A ello se suma que la magnitud de la pérdida varió en cada ejecución sobre entrada idéntica. Un defecto no determinista no puede detectarse mediante una prueba que compare la salida contra un valor esperado fijo, lo que explica que esta clase de errores permanezca latente durante largos períodos en sistemas en producción.

8.3 El costo real de la concurrencia en esta solución fue la memoria

La versión concurrente presentó un RSS máximo de 158.796 KiB frente a 63.960 KiB de la secuencial, aproximadamente 2,5 veces más, y un VSZ cercano a seis veces superior. La causa es estructural: mientras la versión secuencial mantiene en memoria el contenido de un archivo a la vez,

la concurrente mantiene tres simultáneamente, junto con las listas de palabras y los contadores derivados de cada uno.

Ahora bien, ninguna de las dos versiones incrementó su consumo al multiplicar la carga por diez: el RSS permaneció en torno a 64 MB y 159 MB respectivamente, tanto con 10 como con 100 archivos. Esto demuestra que el consumo depende del número de tareas simultáneas en curso y del tamaño individual de los archivos, no del volumen total del lote.

Se concluye que la solución posee memoria acotada y puede procesar lotes arbitrariamente grandes sin agotar los recursos del sistema. Esta propiedad, más que la velocidad, constituye el beneficio arquitectónico efectivamente obtenido, y es consecuencia directa de que el contenido de cada archivo se libere al finalizar su procesamiento en lugar de acumularse en una estructura global.

8.4 La delegación de la sincronización a estructuras especializadas reduce la superficie de error

Los tres recursos compartidos de la solución recibieron tratamientos distintos, con resultados dispares en cuanto a esfuerzo de implementación. La cola de trabajo no requirió una sola línea de código de sincronización, pues **queue.Queue** garantiza internamente la atomicidad de sus operaciones; en ninguna corrida se observó que dos trabajadores procesaran el mismo archivo ni que alguno quedara sin procesar. El diccionario de totales y la bitácora, en cambio, exigieron protección explícita, y fue precisamente en el primero donde la omisión del bloqueo produjo el defecto documentado.

Este contraste responde por qué una cola sincronizada permite distribuir trabajo entre consumidores: además de garantizar que cada elemento sea entregado a un único consumidor, proporciona balanceo automático de carga, ya que cada trabajador solicita una nueva tarea solo al concluir la anterior, y ofrece un mecanismo de terminación ordenada mediante el seguimiento de tareas pendientes.

8.5 Limitaciones de la solución

Del análisis del código y de las mediciones se identifican las siguientes limitaciones:

- **Contención en la bitácora.** El archivo de registro se abre y cierra en cada evento bajo un bloqueo compartido, lo que con 100 archivos implica cientos de aperturas serializadas. Es el principal punto de contención de la versión concurrente.
- **Recorrido duplicado del contenido.** La función de análisis particiona el texto dos veces, duplicando el trabajo de CPU en la etapa más costosa, justamente la que el bloqueo global del intérprete impide paralelizar.
- **Ventana de inconsistencia en el traslado.** Los totales se actualizan antes de mover el archivo; si el traslado fallara, este quedaría contabilizado pese a permanecer en el directorio de entrada.
- **Comprobación no atómica de duplicados.** La verificación de existencia del destino y el traslado son operaciones separadas, configurando el patrón tiempo-de-comprobación a tiempo-de-uso. No se manifiesta aquí porque los nombres son únicos.
- **Alcance de las mediciones.** Las pruebas se ejecutaron sobre ARM64 con 4 vCPU, por lo que los tiempos absolutos no son extrapolables a otras plataformas. Todos los archivos de prueba tenían contenido idéntico, de modo que no se evaluó el balanceo ante tamaños heterogéneos.

8.6 Mejoras propuestas

Sustituir hilos por procesos para el cómputo. Dado que la limitación es el bloqueo global del intérprete, emplear **multiprocessing** permitiría aprovechar los cuatro núcleos, a costa de mayor consumo de memoria y de serializar resultados entre procesos. Es la modificación con mayor impacto sobre el tiempo.

Mantener abierto el descriptor de la bitácora, o delegar el registro en el módulo **logging** con una cola intermedia, eliminando las aperturas repetidas.

Unificar el recorrido del contenido, calculando el número de palabras desde la misma partición que alimenta el contador de frecuencias.

Reordenar la contabilización, actualizando los totales solo tras confirmar el traslado exitoso.

Procesar archivos por bloques, leyendo de forma incremental para manejar archivos de cualquier tamaño con consumo constante.

9. Anexos

Anexo A. Repositorio del proyecto

El código fuente, las evidencias de ejecución y los registros completos se encuentran disponibles en el repositorio del equipo, cuya estructura corresponde a la solicitada en el apartado 9 del enunciado:

SO-Lab01-Integrador/

```
|— README.md
|— requirements.txt
|— src/
|   |— procesador.py
|   |— version_secuencial.py
|   |— version_concurrente.py
|   |— version_sincronizacion_experimental.py
|— scripts/
|   |— generar_archivos_entrada.py
|   |— registrar_evidencias.py
|— data/
|   |— entrada/
|   |— procesados/
|   |— errores/
|   |— reportes/
|— logs/
|   |— sistema.log
|— evidencia/
|   |— capturas/
|   |— comandos_ejecutados.txt
|   |— mediciones.md
|   |— resultados/
|— informe.pdf
```

Anexo B. Código fuente

Se incluye el listado íntegro de **src/procesador.py**, que concentra la lógica compartida por las tres versiones, y de los tres módulos de orquestación. El módulo **procesador.py** contiene la clase **Bitacora**, las funciones de resolución de rutas, análisis de texto, procesamiento individual y generación del reporte consolidado.

Anexo C. Comandos ejecutados

El archivo **evidencia/comandos_ejecutados.txt** registra la secuencia completa de comandos empleados durante el laboratorio, agrupados por etapa: preparación del entorno, generación de archivos de entrada, ejecución de cada versión, observación de procesos y medición de memoria y disco.

Anexo D. Capturas de pantalla

D.1 Configuración de la máquina virtual en VirtualBox **capturas/Entorno de ejecucion.png**

D.2 Observación del proceso Python con **ps capturas/evidencia_7_4.png**

Anexo E. Registros de ejecución

Extractos de **logs/sistema.log** correspondientes a las corridas de las versiones secuencial, concurrente y experimental, incluyendo los eventos de incorporación a la cola, confirmación de procesamiento y cierre de ejecución.

Anexo F. Mediciones completas

El archivo **evidencia/resultados/<marca_temporal>/mediciones.json** contiene el registro estructurado de cada corrida: comando ejecutado, código de salida, estado de memoria antes y después, salida de **/usr/bin/time**, muestras periódicas de **ps**, contenido del reporte consolidado y validación automática de los totales frente a los valores esperados.